

PRO CUP PADEL

Sistema de Gestión de Torneos de Pádel



Resumen Técnico Completo del Sistema

Arquitectura · Módulos · Algoritmos · Base de Datos · Deploy

Autores: Cepeda & Wortley

Proyecto de Tesis Universitaria

Julio 2026

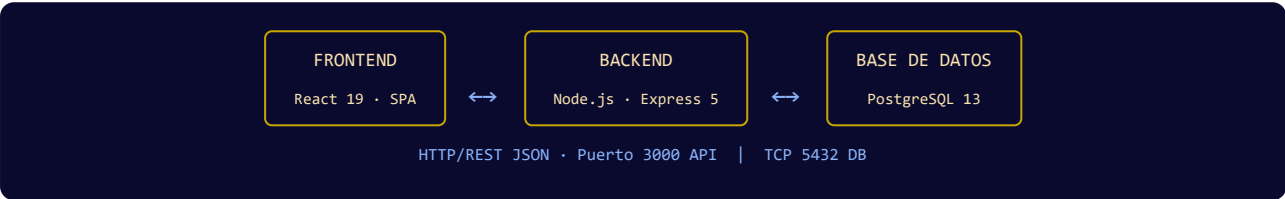
1. Visión General del Sistema

1.1 ¿Qué es Pro Cup Padel?

Pro Cup Padel es una aplicación web full-stack para la gestión integral de torneos de pádel en modalidad dobles/parejas. Cubre el ciclo completo: inscripción de jugadores y equipos, generación de grupos y fixture, registro de resultados por sets, bracket de playoff automático, ranking acumulado histórico y notificaciones por email.

Problema que resuelve: Los torneos de pádel se gestionan manualmente con planillas y WhatsApp. Pro Cup Padel automatiza todos estos procesos en una sola plataforma web accesible desde cualquier dispositivo.

1.2 Arquitectura General



1.3 Stack Tecnológico

Capa	Tecnología	Versión	Propósito
Frontend	React	19	Interfaz SPA (Single Page Application)
Frontend Routing	React Router	7	Navegación entre páginas sin recarga
HTTP Cliente	Axios	latest	Llamadas a la API REST del backend
Gráficos	Recharts	latest	Dashboard con estadísticas visuales
Backend	Node.js + Express	18 + 5.1	API REST, lógica de negocio
Módulos JS	ESM (<code>type:module</code>)	—	Sintaxis moderna <code>import/export</code>
Base de datos	PostgreSQL	13	Persistencia relacional con transacciones
Driver DB	node-postgres (<code>pg</code>)	8.16	Pool de conexiones a la DB
Autenticación	JWT + bcrypt	—	Tokens de sesión + hash de contraseñas
Email	Resend SDK	6.16	Notificaciones automáticas por email
Tareas programadas	node-cron	4.5	Recordatorios 24h antes del torneo
Archivos	Multer	2.0	Subida de fotos de perfil
Contenedores	Docker + Compose	—	Entorno reproducible local
Deploy	Railway	—	Hosting en la nube (producción)

1.4 Roles de usuario

Rol	Cómo se obtiene	Permisos
invitado	Entrar sin cuenta	Ver home, ranking y torneos (solo lectura)
jugador	Registrarse en <code>/registro</code>	+ Inscribirse en torneos, editar perfil
organizador	Registrarse en <code>/registro-admin</code> con token secreto	+ Crear torneos, cargar resultados, gestionar jugadores, ver dashboard

2. Base de Datos — PostgreSQL

TABLA categoria

Almacena las divisiones del deporte (5ta, 4ta, 3ra, etc.). El campo `valor_numerico` (INT 2–8) es el eje del sistema: se usa para validar inscripciones en torneos "suma" y para agrupar el ranking histórico. **No se usa `id_categoria` para el ranking — solo el valor numérico.**

Campo	Tipo	Descripción
id_categoria	SERIAL PK	Identificador único
nombre	VARCHAR	Nombre de la categoría (ej: "5ta")
valor_numerico	INT	Valor 2–8 para validaciones y ranking

TABLA jugador

Tabla central de usuarios del sistema. Un mismo registro sirve tanto para jugadores como organizadores (diferenciados por `rol`). La contraseña siempre se almacena hasheada con bcrypt.

Campo	Tipo	Descripción
id_jugador	SERIAL PK	Identificador único
nombre_jugador / apellido_jugador	VARCHAR	Nombre y apellido
apodo	VARCHAR NULL	Apodo opcional (aparece en nombre de equipo)
email	VARCHAR UNIQUE	Email (también es el usuario de login)
password	VARCHAR	Hash bcrypt de la contraseña
rol	VARCHAR	'jugador' o 'organizador'
categoria_id	FK → categoria	Categoría del jugador
foto_perfil	VARCHAR NULL	Path relativo a la imagen subida

TABLA equipo

Representa una pareja de jugadores. El `nombre_equipo` se genera automáticamente al inscribirse, combinando apellidos/apodos en orden alfabético (ej: "García/López").

Campo	Tipo	Descripción
id_equipo	SERIAL PK	Identificador único
jugador1_id / jugador2_id	FK → jugador	Los dos integrantes de la pareja
nombre_equipo	VARCHAR	Nombre auto-generado ("Apellido1/Apellido2")

TABLA torneo

Almacena la configuración completa de un torneo. Soporta dos formatos de categoría y dos modalidades de juego.

Campo	Tipo	Descripción
id_torneo	SERIAL PK	Identificador único
nombre_torneo	VARCHAR	Nombre del torneo
fecha_inicio / fecha_fin	DATE	Duración del torneo
fecha_cierre_inscripcion	TIMESTAMP	Límite para inscribirse (guardado a las 23:59:59)
maxEquipos	INT	Capacidad máxima de parejas
formato_categoria	VARCHAR	'categoria_fija' o 'suma'
categoria_id	FK → categoria NULL	Solo para formato 'categoria_fija'
suma_categoria	INT NULL	Solo para formato 'suma' (ej: 9 = 5ta+4ta)
modalidad	VARCHAR	'fin_de_semana' o 'liga'
dias_juego	VARCHAR NULL	Para liga: "Lunes,Miércoles"

TABLA inscripcion · grupos · equipos_grupo · partidos_grupo

Tabla	Propósito	Campos clave
inscripcion	Qué equipo está en qué torneo	id_equipo (CASCADE), id_torneo (CASCADE)
grupos	Grupos de la fase (ej: "Grupo A")	id_grupo, id_torneo, nombre
equipos_grupo	Estadísticas por equipo en cada grupo	grupo_id, equipo_id, puntos, partidos_jugados, sets_favor/contra, games_favor/contra
partidos_grupo	Partidos de la fase de grupos con sets y fecha	grupo_id, equipo1_id, equipo2_id, set1..set3 ×2, estado, fecha

TABLA CLAVE partidos_llave — Bracket Playoff

La tabla más compleja. `next_match_id` y `next_slot` forman una cadena que conecta los partidos del bracket: cuando un equipo gana, el sistema usa estos campos para colocarlo automáticamente en el partido siguiente.

Campo	Tipo	Descripción
id_torneo	FK → torneo	Torneo al que pertenece
ronda	VARCHAR	'OCTAVOS' 'CUARTOS' 'SEMIS' 'FINAL'
orden	INT	Posición dentro de la ronda
equipo1_id / equipo2_id	FK → equipo NULL	Participantes (NULL = BYE)
ganador_id	FK → equipo NULL	Se llena al registrar el resultado
next_match_id	FK self-ref NULL	Partido al que avanza el ganador
next_slot	INT NULL	1 o 2: posición del ganador en el siguiente partido
set1..set3 equipo1/2	INT NULL	Resultado de cada set
estado	VARCHAR	'no_iniciado' 'finalizado'

TABLA ranking_jugador · audit_log_ingresos

⚠ **Punto crítico:** El campo `categoria` en `ranking_jugador` almacena el `valor_numerico` (entero 2–8), NO el `id_categoria`. Esto permite que el ranking funcione correctamente en torneos de tipo "suma".

Tabla	Campos clave
<code>ranking_jugador</code>	jugador_id, nombre, apellido, ultima_pareja, torneo_participado, fase_llegada, puntos, categoria (INT = valor_numerico)
<code>audit_log_ingresos</code>	jugador_id, nombre, apellido, ip, user_agent, exitoso (bool), motivo, timestamp

3. Módulos del Backend

backend/src/ — Node.js + Express 5.1 (ESM)
ENTRY POINT app.mjs

Express CORS dotenv node-cron Static files

Punto de entrada de toda la aplicación backend. Hace tres cosas principales:

1. **Configura Express:** activa CORS (permite solicitudes desde cualquier origen), parseo de JSON y sirve la carpeta `uploads/` como archivos estáticos para que el frontend acceda a las fotos de perfil con `/uploads/perfiles/foto.jpg`.
2. **Monta todas las rutas** bajo el prefijo `/api`: playoff, rutas principales, auditoría, auth, ranking, torneos y dashboard.
3. **Programa el cron job** con `node-cron` (expresión `'0 * * * *'` = cada hora en el minuto 0). El job busca torneos cuya `fecha_inicio` esté entre 23 y 25 horas en el futuro y envía emails de recordatorio a todos los jugadores inscriptos. Un `Set` en memoria evita enviar el mismo email dos veces en la misma sesión del proceso.

CONFIG config/db.js

PostgreSQL Pool de conexiones

Crea y exporta un **pool de conexiones** a PostgreSQL usando la librería `pg`. Un pool reutiliza conexiones abiertas en lugar de abrir una nueva por cada query, lo que mejora el rendimiento significativamente. Las credenciales se leen de variables de entorno: `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`. En producción (Railway), estas las provee automáticamente la variable `DATABASE_URL`.

Todas las rutas importan este pool y lo usan con `pool.query()` para consultas simples o `pool.connect()` para obtener un cliente que permite transacciones (`BEGIN / COMMIT / ROLLBACK`).

CONFIG config/multer.js

Multer Upload de archivos Validación MIME

Configura **Multer** para recibir fotos de perfil. Define tres aspectos clave:

- **Destino:** guarda los archivos en `uploads/perfiles/` (la crea si no existe).
- **Nombre único:** genera el nombre con timestamp + número aleatorio para evitar colisiones entre archivos (`perfil-1720000000-123456789.jpg`).
- **Filtro de tipo:** solo acepta JPEG, JPG, PNG y WebP (valida tanto extensión como MIME type). Si el archivo no pasa, rechaza con error. Límite de 5 MB por archivo.

Rutas del Backend

RUTA routes/index.js — Rutas principales

Login Registro Jugadores Torneos Inscripción Grupos Partidos

El archivo de rutas más grande. Concentra toda la lógica CRUD del negocio principal:

Endpoint	Descripción
POST /registro	Registra un jugador (valida passwords, hashea con bcrypt, guarda categoría)
POST /registro-organizadores	Registra organizador. Requiere <code>admin_token</code> que debe coincidir con <code>ADMIN_SECRET</code> del entorno
POST /login	Autentica usuario, genera JWT, registra auditoría, devuelve <code>{ok, token, jugador}</code>
GET/POST/PUT/DELETE /jugadores	CRUD completo. El PUT acepta foto con Multer y construye query dinámica solo con los campos enviados
POST /inscripcion	Inscribe equipo con 5 validaciones encadenadas. Envía email de confirmación asíncrono
POST /torneos/:id/generar-grupos	Genera grupos + fixture automático según modalidad (liga o fin_de_semana)
PUT /partidos-grupo/:id	Carga resultado, actualiza estadísticas. Si todos los partidos finalizaron, auto-genera playoff

Validaciones de inscripción (en orden): ① cierre de inscripción no vencido → ② ningún jugador en torneo con fechas superpuestas → ③ ningún jugador ya inscripto en este torneo → ④ misma pareja no inscripta → ⑤ categoría válida (fija o suma).

RUTA routes/auth.js — Middleware JWT

JWT Middleware Authorization header

Exporta el middleware `requireAuth` que protege las rutas sensibles. Extrae el token del header `Authorization: Bearer <token>`, lo verifica con `jwt.verify()` y, si es válido, adjunta `req.user = { id, role }` para que el siguiente handler sepa quién es el usuario. Si el token es inválido o expiró (6h), devuelve 401.

```
export function requireAuth(req, res, next) {
  const token = req.headers.authorization?.slice(7); // "Bearer xxx"
  const payload = jwt.verify(token, process.env.JWT_SECRET);
  req.user = { id: payload.id, role: payload.role };
  next(); // continúa al siguiente middleware/handler
}
```

CONTROLLER controllers/authController.js

Login alternativo Diagnóstico IP real bcrypt dual

Controlador de login alternativo (montado a través de `auth.js`). A diferencia del login en `routes/index.js`, este controlador incluye herramientas de diagnóstico:

- **IP real:** lee el header `x-forwarded-for` para obtener la IP real del cliente detrás de proxies (Railway usa proxies).
- **Logs de diagnóstico:** imprime en consola el body recibido y la base de datos activa (`SELECT current_database()`) para facilitar el debugging en producción.
- **Verificación dual de contraseña:** detecta si el password almacenado es un hash bcrypt (`startsWith('$2')`) o texto plano (para compatibilidad con datos de prueba), y usa `bcrypt.compare()` o comparación directa según corresponda.
- **JOIN con categoría:** incluye `valor_numerico` de la categoría del jugador en la respuesta para que el frontend lo tenga disponible.

RUTA routes/playoffRoutes.js

Bracket

GET/POST/PUT playoff

Auto-avance

Tres endpoints para el bracket eliminatorio:

- `POST /torneos/:id/playoff` — Llama a `generarPlayoffsSiNoExiste()`. Es **idempotente**: si ya existe, no lo regenera.
- `GET /torneos/:id/playoff` — Devuelve todos los partidos agrupados por ronda en `{OCTAVOS:[...], CUARTOS:[...], SEMIS:[...], FINAL:[...]}`. Incluye JOIN con fotos de los jugadores.
- `PUT /torneos/:id/playoff/partidos/:idPartido` — Carga resultado, calcula el ganador por sets y **lo avanza automáticamente** al siguiente partido mediante `next_match_id` y `next_slot`.

RUTA routes/torneosRoutes.js

Ranking post-torneo

Limpiar grupos

Upsert

Anti-duplicado

Contiene la **versión definitiva** del algoritmo de generación de ranking y rutas de mantenimiento:

- `POST /torneos/:id/generar-ranking` — Función `generarRankingTorneo(client, torneoId)` exportable: verifica que el playoff esté completo, detecta campeón/subcampeón desde la FINAL, calcula la fase de cada equipo, y hace *upsert* en `ranking_jugador`. Tiene **protección anti-duplicado**: si el torneo ya fue procesado, retorna sin volver a insertar. Diseñada para compartir una transacción externa.
- `GET /ranking` — Devuelve el ranking filtrado por `categoria` (valor numérico), ordenado por puntos descendente.
- `DELETE /torneos/:id/limpiar-grupos` — Elimina en cascada todos los datos generados de un torneo (partidos de llave → partidos de grupo → equipos_grupo → grupos) en una sola transacción. Permite reiniciar un torneo desde cero.

RUTA routes/rankingRoutes.js

Ranking manual

Puntos por fase

Versión alternativa del endpoint de ranking (expuesta también en la API). Complementa a `torneosRoutes.js`. Contiene la función `puntosPorFase(fase)` que mapea el nombre de fase a puntos:

Fase	Puntos	Fase	Puntos
Campeón	2000	Cuartos de final	360
Subcampeón	1200	Octavos de final	180
Semifinal	720	Fase de grupos	0

RUTA routes/auditoria.js

Auditoría

Logs

Export CSV

Tres endpoints para consultar los registros de auditoría de login:

- `GET /ingresos` — Consulta `audit_log_ingresos` con filtros opcionales por `jugadorId`, `desde` y `hasta`. Devuelve últimos 100 registros por fecha.
- `GET /ingresos/export` — Descarga directa del CSV de auditoría (`Content-Disposition: attachment`) vía stream.
- `GET /ingresos/test-write` — Endpoint de diagnóstico para verificar que el sistema de logging a archivos funcione en el servidor.

ruta routes/dashboardRoutes.js

Dashboard

KPIs

Estadísticas

Alertas

Cuatro endpoints bajo `/api/dashboard/` para el panel de administración:

- `/kpis` — Totales: cantidad de jugadores, torneos e inscripciones activas.
- `/stats` — Participación desglosada por categoría (datos para los gráficos Recharts).
- `/alerts` — Torneos con cierre de inscripción en las próximas 48 horas.
- `/torneos-status` — Estado de todos los torneos (activos, finalizados, con inscripciones abiertas).

Utilidades del Backend

UTIL utils/generarGrupos.js

Algoritmo

Distribución aleatoria

Casos especiales

Exporta `generarGruposAleatorios(equipos)`. Primero mezcla aleatoriamente los equipos (`Math.random() - 0.5`) y luego los distribuye según la cantidad total:

Equipos	Distribución	Fase resultante
2	1 grupo de 2	Final directa
4	1 grupo de 4	Semis directas
6–7	2 grupos parejos (3-3 o 4-3)	Semis o cuartos
8	2 grupos de 4	Semis
10–12	3 grupos (4-4-2 o 4-4-3)	Cuartos
14–15	4 grupos (4-4-3-3 o similar)	Cuartos
16	4 grupos de 4	Cuartos
32	8 grupos de 4	Octavos

La función `cortarParejo(lista, n)` distribuye los equipos de a uno por grupo en modo rotativo, garantizando que la diferencia entre grupos sea como máximo 1 equipo.

UTIL utils/generarPlayoff.js

Algoritmo

Snake-seeding

BYE

next_match_id

Implementa `generarPlayoffSiNoExiste(torneoId)`. Algoritmo en 6 pasos:

1. **Extrae top 2 de cada grupo** ordenado por puntos → sets a favor → games a favor.
2. **Snake-seeding**: 1ro Grupo A vs 2do Grupo B, 1ro Grupo B vs 2do Grupo A, etc. Evita que los mejores se enfrenten antes de la final.
3. **Determina rondas** por potencia de 2: 4 equipos → SEMIS+FINAL, 8 → CUARTOS+SEMIS+FINAL, 16 → OCTAVOS+...
4. **Asigna BYE** a los mejores seeds si el número no es potencia exacta de 2.
5. **Casos especiales**: 3 grupos (6 equipos) → hardcoded con play-in. 1 grupo → va directo a final/semis/cuartos.
6. **Encadena los partidos**: guarda `next_match_id` y `next_slot` para que el auto-avance funcione sin intervención.

UTIL utils/mailer.js

Resend SDK

Email HTML

Lazy init

Resiliente

Encapsula el envío de emails con **Resend SDK**. El cliente se inicializa de forma *lazy*: solo si `RESEND_API_KEY` existe en el entorno, evitando crash en desarrollo sin credenciales. Si la variable no existe, las funciones retornan silenciosamente sin error.

```
const resend = process.env.RESEND_API_KEY
  ? new Resend(process.env.RESEND_API_KEY)
  : null; // sin clave → no crash, solo no envía
```

Exporta dos funciones: `enviarConfirmacionInscripcion()` (al inscribirse) y `enviarRecordatorioTorneo()` (invocado por el cron job). Ambos usan HTML inline con el estilo de Pro Cup Padel.

UTIL utils/logIngreso.js + utils/fileAudit.js

Auditoría dual

CSV + JSONL

Promise.allSettled

`logIngreso.js` es el punto central de auditoría. Se llama desde cada resultado posible del login (éxito, usuario no encontrado, contraseña incorrecta, error interno). Escribe en dos destinos independientes:

1. Tabla `audit_log_ingresos` en PostgreSQL (dentro de try/catch propio).
2. Archivos de log vía `fileAudit.js`.

`fileAudit.js` mantiene dos archivos en `./logs/`: un CSV y un JSONL (una línea JSON por evento). Usa `Promise.allSettled()` para escribir ambos en paralelo sin que el fallo de uno afecte al otro. Si el directorio no existe, lo crea automáticamente.

4. Módulos del Frontend

frontend/src/ — React 19 + React Router 7

ENTRY POINT index.js

React DOM

Axios baseURL

AuthProvider

BrowserRouter

Punto de entrada del frontend. Hace dos cosas antes de renderizar la app:

1. **Configura el `baseURL` global de Axios:** `process.env.REACT_APP_API_URL || 'http://localhost:3000'`. En producción usa la URL del backend de Railway; en local, `localhost:3000`. Esta es la única fuente de verdad para la URL del backend.
2. **Envuelve la app** en `AuthProvider` (contexto de usuario) y `BrowserRouter` (router).

⚠ Importante: `REACT_APP_API_URL` debe apuntar al backend (`b7ce`), NO al frontend (`6cb8`). En Railway se configura como variable de build del servicio frontend.

APP App.js

Rutas ProtectedRoute Axios interceptors JWT automático

Define el árbol de rutas completo y configura Axios globalmente con dos interceptors:

- **Request interceptor:** lee el token JWT del `localStorage` y lo adjunta automáticamente en el header `Authorization: Bearer <token>` en cada llamado. Ningún componente necesita manejar el token manualmente.
- **Response interceptor:** captura globalmente errores 401 (no autenticado) y 403 (sin permisos).

ProtectedRoute: componente wrapper que verifica si hay usuario logueado en `localStorage` y si su rol está en la lista `allow[]`. Si no, redirige al login o al home del rol.

LAYOUT layout/LayoutBase.js

Layout Navbar condicional useLocation

Wrapper que envuelve **todas las páginas** de la aplicación. Usa `useLocation()` de React Router para conocer la ruta actual. Renderiza el `Navbar` en todas las páginas **excepto** en `/` (login) y `/registro`, donde no tiene sentido mostrarlo. El contenido de cada página va dentro de `<main className="layout-content">`.

```
export default function LayoutBase({ children }) {
  const { pathname } = useLocation();
  const hide = pathname === '/' || pathname === '/registro';
  return (
    <>
      {!hide && <Navbar />}
      <main className="layout-content">{children}</main>
    </>
  );
}
```

CONTEXT context/AuthContext.js

React Context Estado global localStorage Sin prop-drilling

Provee el estado del usuario autenticado a toda la aplicación sin necesidad de pasar props manualmente entre componentes (problema conocido como *prop drilling*). Al iniciarse, lee el usuario guardado en `localStorage` para que la sesión persista entre recargas de página. Expone:

- `jugador` — objeto con id, nombre, apellido, email, role y token del usuario logueado.
- `setJugador()` — función para actualizar el estado global (usada en Login al autenticarse).

Cualquier componente puede acceder al usuario con el hook `useAuth()` sin importar en qué nivel del árbol esté.

Páginas de Autenticación**PÁGINA** pages/Login.js — /

Público JWT Redirección por rol

Formulario de inicio de sesión. Envía email + contraseña a `POST /api/login`. Si el login es exitoso, guarda el token y datos del usuario en `localStorage` y en el contexto `AuthContext`, luego redirige según el rol: organizador → `/home-organizador`, jugador → `/home-jugador`. Muestra mensajes de error específicos por código HTTP (401, 404). También permite entrar como invitado sin credenciales.

PÁGINA pages/Registro.js — /registro

Público Rol: jugador Categoría dinámica

Formulario de auto-registro para jugadores. Incluye selector de categoría que se carga dinámicamente desde `GET /api/categorias`. Valida localmente que las contraseñas coincidan antes de enviar. Al registrarse exitosamente, redirige al login. Tiene botón "Volver al Login".

PÁGINA pages/RegistroOrganizador.js — /registro-admin

Semipúblico Token secreto Rol: organizador

Formulario para crear una cuenta de organizador. Requiere el token secreto (`admin_token`) configurado en el backend como `ADMIN_SECRET`. La ruta es pública (no requiere login previo) pero el token la protege: si es incorrecto, el backend devuelve 403. El token por defecto es `procup2026`.

Páginas del Organizador

PÁGINA pages/CrearTorneo.js — /crear-torneo

Organizador Formulario dinámico

Formulario para crear un nuevo torneo con campos condicionales: si el `formato_categoria` es "categoría fija" muestra selector de categoría; si es "suma" muestra campo numérico. Si la modalidad es "liga", aparece el selector de días de juego. Llama a `POST /api/torneos`.

PÁGINA pages/CargarResultado.js — /cargar-resultado

Organizador Validación de sets Grupos + Playoff

Permite cargar los resultados de un partido en dos modos: fase de grupos (`PUT /api/partidos-grupo/:id`) y playoff (`PUT /api/torneos/:id/playoff/partidos/:idPartido`). Valida marcadores válidos de pádel: 6-0 a 6-4, 7-5, 7-6 (tie-break). El backend actualiza estadísticas y, si todos los partidos de grupos están completos, genera el playoff automáticamente.

PÁGINA pages/Dashboard.js — /dashboard

Organizador Recharts KPIs Alertas

Panel de administración. Carga datos de 4 endpoints en paralelo con `Promise.all` y los muestra en: tarjetas de KPIs (totales), gráficos de barra/torta con **Recharts** (participación por categoría), alertas de torneos próximos a vencer y estado de torneos activos.

PÁGINA pages/AdminJugadores.js — /admin/jugadores

Organizador CRUD completo Modal

CRUD completo de jugadores. Muestra tabla con opciones de editar y eliminar. Modal para crear/editar con todos los campos. Carga jugadores y categorías en paralelo al montar. Confirmación antes de eliminar.

PÁGINA pages/AdminInscripcion.js — /admin/inscripcion

Organizador Inscripción manual

Permite al organizador inscribir equipos manualmente: selecciona torneo → elige dos jugadores de la lista de disponibles (filtrando los ya inscritos) → envía inscripción. Útil para cargar datos presencialmente o importar torneos pasados.

PÁGINA `pages/MisTorneosOrganizador.js` — `/organizador/torneos`[Organizador](#) [Hub de gestión](#)

Hub central de operaciones. Desde aquí el organizador puede: ver equipos inscriptos, generar grupos, ver el fixture, regenerar grupos si hay errores, y navegar al bracket de playoff. Es la pantalla más usada durante un torneo activo.

Páginas del Jugador y Públicas

PÁGINA `pages/Inscripcion.js` — `/inscripcion`[Jugador](#) [Verificación previa](#)

Permite inscribirse en un torneo junto con un compañero. Flujo: selecciona torneo → selecciona compañero de la lista de disponibles (sin los ya inscriptos) → verifica previamente si están inscriptos → envía inscripción. El backend valida categorías y superposición de fechas.

PÁGINA `pages/Perfil.js` — `/perfil`[Jugador / Organizador](#) [FormData](#) [Foto de perfil](#)

Edición del perfil personal. La foto se envía como `multipart/form-data` usando `FormData` de JavaScript (no JSON), procesada por Multer en el backend. Permite editar nombre, apellido, apodo, teléfono, contraseña y foto.

PÁGINA `pages/Home.js` — `/home-*`[Todos los roles](#) [Dashboard visual](#) [Carrusel](#)

Página principal post-login. Muestra carruseles de torneos activos/próximos, preview del ranking por categoría y accesos rápidos según el rol del usuario. Carga datos de torneos, categorías y ranking en paralelo.

PÁGINA `pages/Torneos.js` — `/torneosllave`[Todos los roles](#) [Bracket visual](#) [Grupos](#) [Fixture Liga](#)

Vista principal de un torneo. Tres secciones según el estado: equipos inscriptos (antes de grupos), tabla de posiciones y fixture de grupos (durante la fase grupal), y bracket playoff (fase eliminatoria). Para modalidad liga muestra el fixture completo con fechas por jornada.

PÁGINA `pages/Ranking.js` — `/ranking`[Todos los roles](#) [Filtro por categoría](#)

Ranking histórico acumulado con filtro por categoría. Muestra posición, nombre, apellido, pareja, último torneo, fase alcanzada y puntos totales. Ordena por puntos descendente.

Componentes Reutilizables

COMPONENTE pages/Navbar.js

Navegación Dinámico por rol useAuth

Barra de navegación dinámica. Lee el rol del usuario desde `useAuth()` y renderiza opciones distintas para organizador, jugador e invitado. Incluye el botón de cerrar sesión que limpia el `localStorage`, resetea el contexto y redirige al login. Renderizado condicionalmente por `LayoutBase.js` (oculto en login y registro).

COMPONENTE pages/PlayoffBrackets.js

Bracket visual Rondas dinámicas Sets + ganador

Renderiza el bracket eliminatorio. Recibe el objeto `{OCTAVOS, CUARTOS, SEMIS, FINAL}` del backend y muestra cada partido como una tarjeta con: nombres de equipos, fotos de perfil de los jugadores (vía `TeamAvatar`), sets jugados y el ganador resaltado visualmente. Adapta la cantidad de columnas según las rondas del torneo.

COMPONENTE pages/TeamAvatar.js

Avatar Fotos de perfil Fallback con iniciales

Muestra el avatar de un equipo combinando las fotos de perfil de los dos jugadores. Si un jugador no tiene foto subida, muestra un avatar con sus iniciales. Las URLs de las fotos apuntan a `/uploads/perfiles/` servido por Express como estático.

5. Flujo Completo de un Torneo

- [Organizador] Crea el torneo** — Completa el formulario en `/crear-torneo`: nombre, fechas, formato (categoría fija o suma), modalidad (fin de semana o liga) y máximo de equipos. El backend valida y guarda con `POST /api/torneos`.
- [Jugadores] Se registran e inscriben** — Cada jugador se registra en `/registro`. Luego va a `/inscripcion`, elige el torneo y su compañero. El backend valida categorías, fechas superpuestas y envía emails de confirmación a ambos.
- [Organizador] Genera grupos** — Desde `/organizador/torneos` genera los grupos. El sistema distribuye aleatoriamente los equipos y crea el fixture. Para liga, calcula las fechas de cada jornada según los días configurados.
- [Organizador] Carga resultados de grupos** — Desde `/cargar-resultado` ingresa los sets de cada partido. El sistema actualiza la tabla de posiciones. Cuando se carga el último partido pendiente → **el playoff se genera automáticamente**.
- [Organizador] Carga resultados del playoff** — Carga resultado partido por partido. El ganador avanza automáticamente al siguiente encuentro. Al terminar la FINAL → el sistema genera el ranking del torneo automáticamente.
- [Todos] Visualizan resultados y ranking** — Cualquier usuario (incluso invitados) puede ver el bracket en `/torneosllave` y el ranking acumulado en `/ranking`, filtrado por categoría.

6. Seguridad

Aspecto	Implementación	Por qué
Contraseñas	bcrypt con salt factor 10	Irreversible. Aunque roben la DB no obtienen contraseñas en claro
Sesiones	JWT con expiración 6h	Sin estado en servidor, escalable. El token expira automáticamente
Autorización	Middleware por rol (<code>requireAuth</code> , <code>esOrganizador</code>)	Cada endpoint verifica el rol antes de ejecutar
Registro de admins	Token secreto (<code>ADMIN_SECRET</code>)	Impide que cualquiera cree cuentas de organizador
Variables sensibles	Variables de entorno (.env / Railway)	Nunca en el código fuente ni en el repositorio
Subida de archivos	Validación MIME + límite 5MB en Multer	Previene archivos maliciosos y sobrecarga del servidor

7. Deploy en Producción (Railway)

Servicio	Cómo se construye	Variables clave
Backend	Dockerfile en <code>/backend</code> . Arranca con <code>node src/app.mjs</code>	<code>DATABASE_URL</code> , <code>JWT_SECRET</code> , <code>ADMIN_SECRET</code> , <code>RESEND_API_KEY</code>
Frontend	Dockerfile multi-stage: build CRA → sirve con <code>serve</code> . URL del backend bakeada en el bundle.	<code>REACT_APP_API_URL</code> = URL del backend (sin <code>/api</code>)
PostgreSQL	Servicio administrado de Railway	<code>DATABASE_URL</code> generada automáticamente

CI/CD: Al hacer `git push origin optimized-version`, Railway detecta el cambio y rebuilda automáticamente los contenedores sin intervención manual.

8. Preguntas Frecuentes de Defensa

¿Por qué Node.js y no otro lenguaje backend?

JavaScript en el backend permite compartir lógica con el frontend y es ideal para APIs REST por su modelo de I/O no bloqueante (event loop), que maneja eficientemente múltiples conexiones simultáneas sin threads.

¿Por qué PostgreSQL y no MongoDB?

Los datos son fuertemente relacionales: jugadores → equipos → inscripciones → grupos → partidos. El modelo relacional con FK y transacciones garantiza integridad, especialmente en operaciones como inscripción donde varios inserts deben ser atómicos (todos o ninguno).

¿Cómo funciona JWT?

Token con tres partes en base64: header (algoritmo), payload (id y rol del usuario) y firma (HMAC-SHA256 con `JWT_SECRET`). El backend verifica la firma en cada request sin consultar la DB, lo que hace la autenticación muy eficiente y sin estado.

¿Cómo se evita que dos jugadores se inscriban en torneos superpuestos?

La ruta `POST /api/inscripcion` hace una query SQL que busca si alguno de los dos jugadores tiene inscripciones en otros torneos cuyos rangos de fechas se solapan con el torneo nuevo: `NOT (fecha_fin < fecha_inicio_nuevo OR fecha_inicio > fecha_fin_nuevo)`.

¿Qué es Docker Compose y por qué se usa?

Permite definir en un YAML los 3 servicios (DB, backend, frontend) con sus configs y redes. Con `docker-compose up --build` se levanta todo el sistema en cualquier máquina sin instalar dependencias manualmente, garantizando reproducibilidad.

¿Por qué `valor_numerico` y no `id_categoria` en el ranking?

Porque los torneos de tipo "suma" no tienen categoría fija: dos jugadores de distinta categoría juegan juntos (ej: 5ta+4ta=SUMA9). El `valor_numerico` individual es el único denominador común para agrupar el ranking coherentemente entre ambos tipos de torneo.

¿Cómo funciona el auto-avance en el playoff?

Al crear el bracket, cada partido de playoff tiene `next_match_id` (FK al partido siguiente) y `next_slot` (1 o 2). Al cargar el resultado, el sistema calcula el ganador y ejecuta: `UPDATE partidos_llave SET equipo{slot}_id = ganador_id WHERE id = next_match_id`.

¿Por qué React Context y no Redux?

El estado compartido del sistema es simple: solo el usuario autenticado. Context es suficiente y evita la complejidad de Redux (actions, reducers, store). Redux se justificaría en apps con múltiples fuentes de estado complejo y sincronización frecuente.

¿Qué pasa si el servidor de email no está configurado?

El sistema es resiliente: el cliente Resend se inicializa solo si `RESEND_API_KEY` está definida. Si no, las funciones retornan silenciosamente sin error. El flujo de inscripción y login completa igual.